

LAPORAN PRAKTIKUM ANALISIS KOMPLEKSITAS ALGORITMA



Disusun Oleh:

Nama : Naula Rizwana
Nim : 2025573010058
Kelas : TI-1C
Dosen : Muhammad Reza Zulman, S.ST., M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

PRODI TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2025-2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dan Algoritma, khususnya pada Modul 5 yang membahas mengenai Analisis Kompleksitas Algoritma.

Dalam dunia pemrograman, kemampuan untuk menulis kode yang berjalan dengan benar saja tidaklah cukup. Seorang programmer juga dituntut untuk mampu menghasilkan algoritma yang efisien, baik dari segi waktu eksekusi maupun penggunaan memori. Oleh karena itu, pemahaman mengenai Big O Notation menjadi sangat penting sebagai alat untuk mengukur dan membandingkan performa algoritma.

Melalui praktikum ini, penulis mempelajari bagaimana cara menganalisis kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity), serta bagaimana membandingkan dua algoritma yang berbeda untuk menentukan mana yang lebih optimal. Selain itu, praktikum ini juga memberikan pengalaman langsung dalam menguji performa algoritma menggunakan JavaScript dan Node.js.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Akhir kata, semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai pentingnya analisis kompleksitas dalam pengembangan perangkat lunak.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
BAB II DASAR TEORI.....	2
2.1 Pengertian Analisis Kompleksitas Algoritma.....	2
2.2 Big O Notation.....	2
2.3 Kelas-Kelas Kompleksitas Big O.....	2
2.4 Aturan Penyederhanaan Big O	3
2.5 Time Complexity	3
2.6 Space Complexity.....	4
2.7 Time-Space Trade-Off.....	4
2.8 Pentingnya Analisis Kompleksitas	4
BAB III METODOLOGI PRAKTIKUM	5
3.1 Langkah Kerja	5
Bagian 1 — Intro Kompleksitas	5
Bagian 2 — Notasi Big O.....	6
Bagian 3 — Space Complexity.....	9
Bagian 4 — Mengenali Pola Kompleksitas.....	11
3.2 Tugas Praktikum	12
BAB IV KESIMPULAN.....	15

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan teknologi informasi yang semakin pesat, kebutuhan akan aplikasi yang cepat dan efisien menjadi sangat penting. Setiap program yang dibuat tidak hanya dituntut untuk berjalan dengan benar, tetapi juga harus mampu menangani data dalam jumlah besar dengan waktu yang optimal dan penggunaan memori yang efisien.

Seringkali, sebuah algoritma yang terlihat sederhana dapat menjadi sangat lambat ketika digunakan pada data berskala besar. Hal ini disebabkan oleh kompleksitas algoritma yang tidak efisien. Oleh karena itu, diperlukan suatu metode untuk menganalisis dan mengukur performa algoritma, yaitu melalui konsep Big O Notation.

Big O Notation digunakan untuk menggambarkan seberapa besar pertumbuhan waktu eksekusi atau penggunaan memori suatu algoritma terhadap ukuran input. Dengan memahami konsep ini, seorang programmer dapat memilih algoritma yang lebih baik serta menghindari penggunaan algoritma yang memiliki kompleksitas tinggi seperti $O(n^2)$ atau $O(n^3)$.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami pentingnya analisis kompleksitas algoritma serta mampu menerapkannya dalam pemrograman menggunakan JavaScript dan Node.js.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Memahami konsep Big O Notation
- Menganalisis kompleksitas waktu dan ruang
- Menentukan kompleksitas dari suatu algoritma
- Membandingkan performa algoritma
- Mengidentifikasi algoritma yang tidak efisien
- Menerapkan analisis kompleksitas dalam kode program

BAB II DASAR TEORI

2.1 Pengertian Analisis Kompleksitas Algoritma

Analisis kompleksitas algoritma merupakan suatu metode yang digunakan untuk mengukur tingkat efisiensi suatu algoritma dalam menyelesaikan masalah. Efisiensi tersebut dilihat dari dua aspek utama, yaitu waktu eksekusi (time complexity) dan penggunaan memori (space complexity). Analisis ini sangat penting karena sebuah algoritma yang berjalan dengan benar belum tentu efisien ketika digunakan pada data dalam jumlah besar.

Dalam praktiknya, algoritma yang memiliki kompleksitas tinggi akan membutuhkan waktu yang lebih lama dan sumber daya yang lebih besar. Oleh karena itu, analisis kompleksitas digunakan untuk membandingkan beberapa algoritma dan memilih yang paling optimal. Hal ini menjadi sangat penting terutama dalam pengembangan sistem berskala besar seperti aplikasi web, sistem database, dan aplikasi berbasis data lainnya.

2.2 Big O Notation

Big O Notation adalah notasi matematika yang digunakan untuk menggambarkan laju pertumbuhan (growth rate) suatu algoritma terhadap ukuran input (n). Big O tidak mengukur waktu secara langsung dalam satuan detik, melainkan menggambarkan bagaimana waktu atau memori bertambah seiring dengan bertambahnya jumlah data.

Big O berfokus pada kondisi terburuk (worst-case scenario), sehingga dapat memberikan gambaran umum mengenai performa algoritma dalam kondisi paling tidak menguntungkan. Dengan menggunakan Big O, programmer dapat membandingkan efisiensi algoritma tanpa bergantung pada perangkat keras atau bahasa pemrograman yang digunakan.

2.3 Kelas-Kelas Kompleksitas Big O

Berikut adalah beberapa kelas kompleksitas dalam Big O yang umum digunakan:

- **$O(1)$ (Konstan)**
Kompleksitas ini menunjukkan bahwa waktu eksekusi tidak bergantung pada ukuran input. Contohnya adalah operasi aritmatika sederhana atau akses elemen array berdasarkan indeks.
- **$O(\log n)$ (Logaritmik)**
Kompleksitas ini terjadi ketika setiap langkah algoritma membagi masalah menjadi bagian yang lebih kecil, biasanya menjadi setengahnya. Contoh yang umum adalah algoritma Binary Search.

- **$O(n)$ (Linear)**
Waktu eksekusi bertambah secara linear sesuai dengan jumlah input. Contohnya adalah proses pencarian dalam array menggunakan loop.
- **$O(n \log n)$ (Linearitmik)**
Kombinasi antara linear dan logaritmik, biasanya ditemukan pada algoritma sorting seperti Merge Sort dan Quick Sort.
- **$O(n^2)$ (Kuadratik)**
Terjadi pada algoritma dengan dua perulangan bersarang. Kompleksitas ini sangat lambat untuk data besar, contohnya Bubble Sort.
- **$O(2^n)$ (Eksponensial)**
Kompleksitas ini sangat tidak efisien karena jumlah operasi meningkat secara eksponensial. Contohnya adalah algoritma rekursif Fibonacci tanpa optimasi.

2.4 Aturan Penyederhanaan Big O

Dalam penulisan Big O, terdapat beberapa aturan penyederhanaan, yaitu:

1. Mengabaikan konstanta
Contoh: $O(2n)$ disederhanakan menjadi $O(n)$
2. Mengabaikan suku yang lebih kecil
Contoh: $O(n^2 + n)$ menjadi $O(n^2)$
3. Mengambil suku yang paling dominan
Karena suku tersebut yang paling berpengaruh ketika n sangat besar

Aturan ini bertujuan untuk mempermudah analisis dan fokus pada faktor yang paling berpengaruh terhadap performa algoritma.

2.5 Time Complexity

Time Complexity adalah ukuran yang digunakan untuk mengetahui berapa lama waktu yang dibutuhkan oleh suatu algoritma dalam menyelesaikan suatu masalah berdasarkan ukuran input.

Sebagai contoh:

- Algoritma dengan satu loop memiliki kompleksitas $O(n)$
- Algoritma dengan nested loop memiliki kompleksitas $O(n^2)$

Semakin besar nilai kompleksitas, maka semakin lama waktu yang dibutuhkan untuk memproses data.

2.6 Space Complexity

Space Complexity adalah ukuran jumlah memori tambahan yang digunakan oleh suatu algoritma selama proses eksekusi. Memori tambahan ini disebut juga sebagai auxiliary space.

Contoh:

- Menggunakan variabel sederhana $\rightarrow O(1)$
- Membuat array baru $\rightarrow O(n)$
- Rekursi $\rightarrow$ menggunakan call stack (bisa $O(n)$)

Analisis space complexity penting karena memori juga merupakan sumber daya yang terbatas.

2.7 Time-Space Trade-Off

Dalam pengembangan algoritma, sering terjadi trade-off antara waktu dan ruang. Artinya, untuk membuat algoritma lebih cepat, sering kali dibutuhkan memori tambahan, dan sebaliknya.

Contoh:

- Menggunakan Set atau Map dapat mempercepat pencarian dari $O(n)$ menjadi $O(1)$
- Namun membutuhkan memori tambahan sebesar $O(n)$

Oleh karena itu, pemilihan algoritma harus mempertimbangkan keseimbangan antara kecepatan dan penggunaan memori.

2.8 Pentingnya Analisis Kompleksitas

Analisis kompleksitas sangat penting karena:

- Membantu memilih algoritma yang efisien
- Menghindari performa buruk pada data besar
- Membantu optimasi program
- Digunakan dalam pengembangan sistem skala besar

Tanpa memahami kompleksitas algoritma, seorang programmer akan kesulitan dalam menilai kualitas kode yang dibuat.

BAB III METODOLOGI PRAKTIKUM

3.1 Langkah Kerja

Buat folder praktikum-4/ dan masuk ke dalamnya dan Inisialisasi npm project.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
● $ mkdir praktikum-5

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
● $ cd praktikum-5

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ npm init -y
Wrote to D:\2025573010058_PraktikumAlgoritmaDanStrukturData\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Bagian 1 — Intro Kompleksitas

Langkah 1 Pastikan kamu berada di dalam folder praktikum-5/. Buat file baru bernama 01-introkompleksitas.js.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ touch 01-intro-kompleksitas.js
```

Langkah 2 Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```
JS 01-intro-kompleksitas.js M X
praktikum-5 > JS 01-intro-kompleksitas.js > ...
1  function ukurWaktu(label, fn) {
2      console.time(label);
3      fn();
4      const selesai = Date.now();
5      console.log(`${label}: ${selesai - mulai} ms`);
6  }
7  const N = 100_000; // ukuran data: 100 ribu
8  function jumlahkanLinear(n) {
9      let total = 0;
10     for (let i = 1; i <= n; i++) total += i;
11     return total;
12 }
13 function jumlahkanRumus(n) {
14     return (n * (n + 1)) / 2;
15 }
16 function cariPasangan(arr) {
17     const pasangan = [];
18     for (let i = 0; i < arr.length; i++) {
19         for (let j = i + 1; j < arr.length; j++) {
20             if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
21         }
22     }
23     return pasangan;
24 }
25 const data = Array.from({ length: 5000 }, (_, i) => i);
26 console.log("=== Perbandingan Waktu Eksekusi ===");
27 ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
28 ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
29 ukurWaktu("O(n²) cariPasangan ", () => cariPasangan(data));
30 console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```


Langkah 3 Simpan file, lalu jalankan dari terminal.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
${label}: ${selesai - mulai} ms
${label}: ${selesai - mulai} ms
${label}: ${selesai - mulai} ms
```

Bagian 2 — Notasi Big O

Langkah 1 Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ touch 02-kelas-big-o.js
```

Langkah 2 Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut

```
praktikum-5 > JS 02-kelas-big-o.js > ...
1 // 02-kelas-big-o.js
2 // // Contoh konkret setiap kelas kompleksitas Big O
3 // // — O(1) — Konstan —————
4 function ambilPertama(arr) {
5   return arr[0];
6 } // O(1)
7 function tambahkanItem(arr, item) {
8   arr.push(item);
9 } // O(1)
10 function isGenap(n) {
11   return n % 2 === 0;
12 } // O(1)
13 // // — O(log n) — Logaritmik —————
14 function binarySearch(arr, target) {
15   let kiri = 0,
16     kanan = arr.length - 1;
17   let langkah = 0;
18   while (kiri <= kanan) {
19     langkah++;
20     const tengah = Math.floor((kiri + kanan) / 2);
21     if (arr[tengah] === target) {
22       console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
23       return tengah;
24     }
25     if (arr[tengah] < target) kiri = tengah + 1;
26     else kanan = tengah - 1;
```

Langkah 3 Simpan file, lalu jalankan dari terminal

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgorit
● $ node 02-kelas-big-o.js
=== O(1) – selalu cepat ===
10
true

=== O(log n) – Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) – Linear Search ===
Max dari 1000 elemen: 998

=== O(n²) – Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2ⁿ) – Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
164 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Buat file baru latihan-1.js.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ touch latihan-1.js
```

```
praktikum-5 > JS latihan-1.js > ...
1 // Fungsi A → O(1)
2 function fungsiA(n) {
3   // Big O: O(1)
4   // Alasan: hanya melakukan 1 operasi (perkalian),
5   // tidak ada perulangan dan tidak bergantung pada ukuran n
6   return n * 2;
7 }
8
9 // Fungsi B → O(n²)
10 function fungsiB(n) {
11   // Big O: O(n²)
12   // Alasan: terdapat 2 loop bersarang
13   // loop luar berjalan n kali
14   // loop dalam juga berjalan n kali
15   // total operasi = n * n = n²
16   for (let i = 0; i < n; i++) {
17     for (let j = 0; j < n; j++) {
18       // console.log(i, j);
19     }
20   }
21 }
22
23 // Fungsi C → O(log n)
24 function fungsiC(n) {
25   // Big O: O(log n)
26   // Alasan: nilai i dikali 2 setiap iterasi
27   // contoh: 1 → 2 → 4 → 8 → ...
28   // jumlah iterasi bertambah secara logaritmik terhadap n
29   for (let i = 1; i < n; i *= 2) {
30     // console.log(i);
31   }
32 }
```

```

31   }
32 }
33
34 // Fungsi D → O(n^3)
35 function fungsiD(n) {
36   let arr = Array.from({ length: n }, (_, i) => i);
37
38   // Big O: O(n^3)
39   // Alasan: terdapat 3 perulangan bersarang (forEach)
40   // masing-masing berjalan sebanyak n elemen
41   // total operasi = n * n * n = n^3
42   arr.forEach((x) => {
43     arr.forEach((y) => {
44       arr.forEach((z) => {
45         // console.log(x, y, z);
46       });
47     });
48   });
49 }
50
51 // Function untuk mengukur waktu eksekusi
52 function hitungKompleksitas(n, fn) {
53   // Mengambil waktu sebelum fungsi dijalankan
54   let start = Date.now();
55
56   // Menjalankan fungsi
57   fn(n);

```

```

51 // Function untuk mengukur waktu eksekusi
52 function hitungKompleksitas(n, fn) {
53   // Mengambil waktu sebelum fungsi dijalankan
54   let start = Date.now();
55
56   // Menjalankan fungsi
57   fn(n);
58
59   // Mengambil waktu setelah fungsi selesai
60   let end = Date.now();
61
62   // Menampilkan lama eksekusi dalam milidetik
63   console.log(fn.name + ":", end - start, "ms");
64 }
65
66 // Menjalankan semua fungsi
67 let n = 1000;
68
69 hitungKompleksitas(n, fungsiA); // sangat cepat (O(1))
70 hitungKompleksitas(n, fungsiB); // lebih lama (O(n^2))
71 hitungKompleksitas(n, fungsiC); // cepat (O(log n))
72
73 // ⚠ Fungsi D sangat berat, jangan pakai n besar
74 hitungKompleksitas(100, fungsiD); // gunakan n kecil saja

```

Jalankan di terminal

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ node latihan-1.js
fungsiA: 0 ms
fungsiB: 4 ms
fungsiC: 0 ms
fungsiD: 5 ms
```

Bagian 3 — Space Complexity

Langkah 1 Buat file baru bernama 03-space-complexity.js di dalam folder praktikum-5/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ touch 03-space-complexity.js
```

Langkah 2 Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut

```
praktikum-5 > JS 03-space-complexity.js > ...
1  function jumlahArray(arr) {
2      let total = 0;
3      for (const x of arr) total += x;
4      return total;
5  }
6
7  function duplikasiArray(arr) {
8      const baru = [];
9      for (const x of arr) baru.push(x * 2);
10     return baru;
11 }
12
13 function faktorialRekursif(n) {
14     if (n <= 1) return 1;
15     return n * faktorialRekursif(n - 1);
16 }
17
18 function faktorialIteratif(n) {
19     let hasil = 1;
20     for (let i = 2; i <= n; i++) hasil *= i;
21     return hasil;
22 }
23
24 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
25
26 console.log("Jumlah array   :", jumlahArray(arr));
27 console.log("Duplikasi array:", duplikasiArray(arr));
28 console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
29 console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
```

Langkah 3 Simpan file, lalu jalankan dari terminal

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

Latihan 3: Class BankAccount

```
40 // Fungsi 1: Cara Lambat (Brute Force)
41 // =====
42 function cariPasanganLambat(arr, target) {
43   // Big O Time: O(n^2)
44   // Alasan: menggunakan 2 loop bersarang → setiap elemen dibandingkan dengan elemen lain
45   // total perbandingan = n * n
46
47   // Big O Space: O(1)
48   // Alasan: tidak menggunakan struktur data tambahan (hanya variabel biasa)
49
50   for (let i = 0; i < arr.length; i++) {
51     for (let j = i + 1; j < arr.length; j++) {
52       if (arr[i] + arr[j] === target) {
53         return [arr[i], arr[j]];
54       }
55     }
56   }
57
58   return null;
59 }
60
61 // =====
62 // Fungsi 2: Cara Cepat (Menggunakan Set)
63 // =====
64 function cariPasanganCepat(arr, target) {
65   // Big O Time: O(n)
66   // Alasan: hanya 1 loop, setiap elemen dicek sekali
```

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
```

Bagian 4 — Mengenali Pola Kompleksitas

Langkah 1 Buat file baru bernama 04-refactor.js di dalam folder praktikum-5/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
● $ touch 04-refactor.js
```

Langkah 2 Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```
JS 04-refactor.js > ...
1 // 05-refactor.js — Refactoring kode berperforma buruk
2 // =====
3 // SKENARIO 1: Cek duplikat dalam array
4 // =====
5 // BURUK:  $O(n^2)$  — nested loop
6 function adaDuplikatLambat(arr) {
7   for (let i = 0; i < arr.length; i++)
8     for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
9   return false;
10 }
11
12 // BAIK:  $O(n)$  — gunakan Set
13 function adaDuplikatCepat(arr) {
14   const seen = new Set();
15   for (const x of arr) {
16     if (seen.has(x)) return true;
17     seen.add(x);
18   }
19   return false;
20 }
21
22 // =====
23 // SKENARIO 2: Frekuensi kemunculan elemen
24 // =====
25
26 // BURUK:  $O(n^2)$  — indexOf di dalam loop
27 function frekuensiLambat(arr) {
28   const unik = [];
29   const hitung = [];
```

Langkah 3 Simpan file, lalu jalankan dari terminal

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
● $ node 04-refactor.js
Duplikat LAMBAT  $O(n^2)$ : 0 ms
Duplikat CEPAT  $O(n)$  : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

3.2 TUGAS PRAKTIKUM

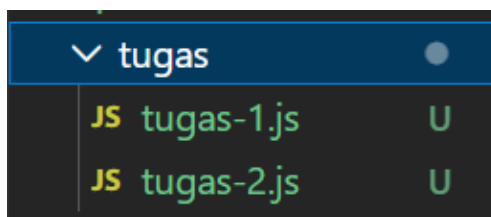
Langkah 1 Buat folder dan file tugas.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
• $ mkdir tugas

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
• $ touch tugas/tugas-1.js tugas/tugas-2.js
```

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
• $ touch tugas-1.js

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
• $ touch tugas-2.js
```



TUGAS 1

```
praktikum-5 > tugas > JS tugas-1.js > ...
1  // =====
2  // TUGAS 1 - ANALISIS & REFACTOR
3  // =====
4
5  // =====
6  // FUNGSI A: INTERSECTION ARRAY
7  // =====
8
9  // Versi Lambat
10 function intersectionLambat(arr1, arr2) {
11     // Big O Time: O(n^2)
12     // Alasan: setiap elemen arr1 dibandingkan dengan semua elemen arr2
13
14     // Big O Space: O(n)
15     // Alasan: menyimpan hasil intersection
16
17     let hasil = [];
18
19     for (let i = 0; i < arr1.length; i++) {
20         for (let j = 0; j < arr2.length; j++) {
21             if (arr1[i] === arr2[j]) {
22                 hasil.push(arr1[i]);
23             }
24         }
25     }
26
27     return hasil;
28 }
```

```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
● $ node tugas-1.js

=== TEST KECIL ===
Intersection Lambat: [ 3, 4, 5 ]
Intersection Cepat: [ 3, 4, 5 ]
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Triplet Lambat: true
Triplet Cepat: false

```

TUGAS 2

```

praktikum-5 > tugas > JS tugas-2.js > ...
1  // =====
2  // FUNGSI-FUNGSI BIG O
3  // =====
4
5  // O(1)
6  function fn_O1(n) {
7    return n + 1;
8  }
9
10 // O(n)
11 function fn_On(n) {
12   let sum = 0;
13   for (let i = 0; i < n; i++) sum += i;
14   return sum;
15 }
16
17 // O(n log n)
18 function fn_OnLogn(n) {
19   let count = 0;
20   let log = Math.log2(n);
21
22   for (let i = 0; i < n; i++) {
23     for (let j = 0; j < log; j++) {
24       count++;
25     }
26   }
27   return count;
28 }

```

```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5/tugas (main)
● $ node tugas-2.js

n = 100
O(1):      0 ms
O(n):      0 ms
O(nlogn):  2 ms
O(n^2):    0 ms

n = 500
O(1):      0 ms
O(n):      0 ms
O(nlogn):  1 ms
O(n^2):    1 ms

n = 1000
O(1):      0 ms
O(n):      0 ms
O(nlogn):  0 ms
O(n^2):    3 ms

n = 5000
O(1):      0 ms
O(n):      0 ms
O(nlogn):  1 ms
O(n^2):    78 ms

```


Langkah terakhir Submit ke GitHub

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main b9177c7] Praktikum 5: Analisis Kompleksitas Algoritma
8 files changed, 670 insertions(+)
create mode 100644 praktikum-5/01-intro-kompleksitas.js
create mode 100644 praktikum-5/02-kelas-big-o.js
create mode 100644 praktikum-5/03-space-complexity.js
create mode 100644 praktikum-5/04-refactor.js
create mode 100644 praktikum-5/latihan-1.js
create mode 100644 praktikum-5/package.json
create mode 100644 praktikum-5/tugas/tugas-1.js
create mode 100644 praktikum-5/tugas/tugas-2.js

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
● $ git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.90 KiB | 1008.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/naularizuana-cell/2025573010058_PraktikumAlgoritmaDanStrukturData.git
```

▼ praktikum-5		
> tugas		
01-intro-kompleksitas.js	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago
02-kelas-big-o.js	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago
03-space-complexity.js	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago
04-refactor.js	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago
latihan-1.js	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago
package.json	Praktikum 5: Analisis Kompleksitas Algoritma	1 minute ago

BAB IV KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan hal yang sangat penting dalam pengembangan program. Big O Notation digunakan sebagai alat untuk mengukur seberapa efisien suatu algoritma dalam menangani data dengan ukuran yang berbeda. Melalui praktikum ini, dapat dipahami bahwa semakin kecil nilai kompleksitas suatu algoritma, maka semakin baik performanya, terutama ketika digunakan pada data dalam jumlah besar. Hasil pengujian menunjukkan bahwa algoritma dengan kompleksitas $O(1)$ memiliki waktu eksekusi yang sangat cepat dan tidak dipengaruhi oleh ukuran input. Sementara itu, algoritma dengan kompleksitas $O(n)$ mengalami peningkatan waktu secara linear, dan algoritma dengan kompleksitas $O(n^2)$ mengalami peningkatan waktu yang sangat signifikan sehingga kurang efisien untuk data besar. Hal ini membuktikan bahwa pemilihan algoritma yang tepat sangat berpengaruh terhadap kinerja program.

Selain itu, praktikum ini juga menunjukkan adanya hubungan antara penggunaan waktu dan memori, di mana algoritma yang lebih cepat sering kali membutuhkan memori tambahan. Oleh karena itu, diperlukan pertimbangan yang tepat dalam memilih algoritma agar dapat mencapai keseimbangan antara efisiensi waktu dan penggunaan ruang. Dengan demikian, pemahaman mengenai Big O Notation, Time Complexity, dan Space Complexity sangat penting bagi mahasiswa dalam menulis kode yang optimal, efisien, dan mampu menangani data dalam skala besar secara efektif.